TF-Photostim Bringup Guide

Full calibration + first photostimulation experiments, step by step. DMD (DLP650LNIR via ViALUX ALP-4.3) + Meadowlark 1K SLM remote focus + CARBIDE @ 1038 nm, Nikon 10x/0.45 default objective, ScanImage + Optotune ETL imaging, substage Basler camera.

Generated 2026-08-15 from the state of the repo at the 3D remote-focusing merge. Every command is real and tested against mocks; items marked ⚠ VERIFY are the on-rig checks that gate trusting the result.

---

0. Roles, links, and safety — read first

The machines

Machine	Runs	Owns
DAQ PC (scope PC, Windows)	MATLAB + this repo	NI PCIe-6323, DMD (ALP DLL), Basler substage camera, all triggers/AO
Imaging PC	MATLAB + ScanImage	2p imaging, Optotune ETL (fastZ), MP-285 (serial, by default)
SLM PC	MATLAB + this repo (clone) + Blink SDK	Meadowlark 1K via PCIe

The network links (docs/PORTS.md is the authority)

Port	Server	Client	What
3043	DAQ PC	imaging PC	stim-metadata control
3044	DAQ PC	imaging PC	live F stream
3045	DAQ PC	imaging PC	ROI centroids (Nx2, or Nx4 with plane tags in 3D)
3046	**SLM PC**	DAQ PC	mask specs + advance commands (`slm_server.m`)
3047	**imaging PC**	DAQ PC	z-motor relay (`si_motor_helper.m`)

Ports 3046/3047 are the two direction-reversed links: those servers must be running **before** the DAQ PC starts anything that needs them.

Safety — the two hardware hazards (both enforced in software, still know them)

1. **DMD ON-fraction cap (50%)** — every 4f relay forms a real air focus at its pupil; an all-ON pattern at full power crosses air breakdown *at any laser power setting you might later raise*. `tfp.hardware.DMD.assertPatternsSafe` refuses any frame > 50% ON on every load, mocks included. Do not work around it.

2. **SLM liquid-crystal alignment cap (44 mW)** — with the 3D-SHOT diffuser gone, the SLM sits at a Fourier plane of the DMD pattern: an all-ON / near-uniform patch focuses each color into a spectral **line** (~0.1 mm²) on the LC. `tfp.util.assertSlmPowerSafe` blocks near-uniform patterns above 44 mW when `slm.enabled` is true. Sparse spot patterns are benign at full field power.

Always begin rig sessions with `tfp.util.safetyChecks('arm')` (the Sequencer checks it per trial), laser at minimum power until Phase 4, and the power meter (Thorlabs PM100D + S350C) physically present.

---

1. Bench + software prerequisites (checklist)

Optics bench (from `docs/optics_handoff.md` — the merged arm)

- [ ] **Lens swaps confirmed installed: f7 = 300 mm, f6 = 80 mm.** ⚠ VERIFY — the committed handoff (rev 4) still describes f7 = 250; the **ratified** build (BOM 2026-08-10) is f7 = 300. Until the optics repo regenerates `docs/optics_handoff.md` (rev ≥ 5), every "expected value" below is provisional and the software warns instead of failing.
- [ ] **3D-SHOT zero-order block at I1 REMOVED** (handoff §6 — where it sits it shadows the middle of the merged field; the SLM zero-order ghost is accepted, not blocked).
- [ ] DMD chip Scheimpflug tilt set — sign is **opposite** the old bring-up-arm instructions (the merged chain adds a fourth inverting relay). Getting it backwards doubles the pattern/pulse split.
- [ ] λ/2 + polarizer set for s-pol on the grating; beam expander identity recorded (custom 1.75× vs GBE02-B changes the dwell-correction table).
- [ ] Thin fluorescent film slide (lateral + z calibration) and thick fluorescent slab (burn/bleach verify) at the rig.

Software

- [ ] Repo cloned + up to date on **all three PCs**; `addpath('src')` in each MATLAB.
- [ ] msocket library on the path of all three (set `msocketPath` config keys).
- [ ] **Optics repo regen** (owner action): in `TF optics simulator`, ratify the f7=300 build and run `python -m configs.dmd_handoff` → commit the new `docs/optics_handoff.md` here. This flips the sanity bands from warn to live.
- [ ] `configs/real.yaml` reviewed: `slm:` (host = SLM PC IP), `zstage:`, `etl:`, `threeD:`, `laser.carbide_modulator_ao_channel` (once wired).
- [ ] Full mock suite green on the DAQ PC (pre-flight): `matlab addpath('src'); results = run(matlab.unittest.TestSuite.fromFolder('tests')); % expect 0 failures (1 "incomplete" = the pending handoff-regen tripwire)`

---

2. SLM PC bringup (the Blink gate) — ~half a day

Work through `docs/blink-api-audit.md` on the SLM PC. Summary:

1. Locate the Meadowlark/Blink install; copy the C wrapper header + SDK manual into `vendor/meadowlark/official/`; record DLL path + SDK version.
2. Fill the audit table (function names ↔ header lines). The load-bearing question: **does this unit do onboard-sequence memory + external-trigger advance?** That decides `slm.trigger_mode`: - YES → `'ttl'` available: wire DAQ `port0/line8` → the controller's trigger input (record polarity/level from the audit). - NO → stay on `'software'` permanently (network `'ADV'`; between-group advances are hundreds of ms apart, latency is immaterial).
3. Replace the `%VENDOR-AUDIT` blocks in `src/+tfp/+hardware/BlinkSLM.m` with the real `calllib` names. **Never invent Blink function names.**
4. Edit `scripts/slm_pc_setup/slm_pc_config.m` (port 3046, DLL/header/LUT paths). Keep `dryRun = true` for the first link test.
5. Start the server and leave it running: `matlab cd <repo>; addpath('src'); addpath('scripts/slm_pc_setup'); slm_server % listens on 3046 until SHUTDOWN / Ctrl-C`

Link test from the DAQ PC (works in dryRun — no hardware touched):

```
slm = tfp.hardware.MeadowlarkSLM(config.slm);   % config = tfp.io.loadConfig('configs/real.ya
slm.connect();                                   % msconnect to SLM PC:3046
sys = tfp.optics.buildDefocusSys(config);        % needs slm.m_relay set — see §7
slm.prepareDefocusSequence([-20 0 20], sys);     % expect 'READY' back
slm.blank();                                     % safe state; expect 'BLANK_OK'
```

Then set `dryRun = false` on the SLM PC, restart `slm_server`, repeat — now the masks really upload. `slm.blank()` (flat mask) is the safe state; use it whenever you walk away.

- [ ] READY handshake works (dryRun)
- [ ] READY handshake works (real upload)
- [ ] BLANK works; decide + record `trigger_mode` in `configs/real.yaml`

---

3. Laser power path — calibrate volts → mW

1. Wire the CARBIDE's external modulator to its DAQ AO channel; set `laser.carbide_modulator_ao_channel` in `configs/real.yaml`.
2. With the power meter at the sample plane, run the existing sweep: `matlab run_powerMeterSweep % scripts/ — writes data/calibration/power_curve_*.mat` and confirm it wrote the calibration path back into the config.
3. Record: max power at sample, the voltage that gives ~5 mW (calibration working power) and ~44 mW (the SLM alignment cap — know where it is on the dial).

- [ ] volts→mW curve saved; `laser.*calibration_file` points at it

---

4. Lateral calibration (2D affines — the existing pipeline)

Target: thin fluorescent film on the stage, substage Basler running (`scripts/basler_live_preview.m` to focus/expose).

4a. DMD → camera (Step A)

```
config = tfp.io.loadConfig('configs/real.yaml');
dmd    = tfp.hardware.DLP650LNIR_DMD(config.dmd);
cam    = tfp.hardware.BaslerSubstageCamera();  cam.initialize(config.camera);
calibA = tfp.calibration.alignDMDtoCamera(dmd, cam);   % 5x5 spot grid, affine fit
```

Good result: `calibA.residualErrorPx < 1` camera px. (Or use `tfp.calibration.calibrationGUI` for the interactive version with per-point reject.)

4b. ScanImage scan field → camera (Step B)

On the imaging PC: ScanImage in **Focus** mode with a **non-square** pixel count (e.g. 512×256 — more pixels on the resonant axis). Then on the DAQ PC:

```
calib = tfp.calibration.crossRegisterScanImage(cam, calibA);   % also composes dmdToScan
```

4c. Verify axis signs (mandatory)

```
calib = tfp.calibration.verifyScanFieldComposition(dmd, calib);
```

Project spot → predicted mROI → operator confirms centering; ≤ 2 attempts resolve the 4 sign combos. Signs land in the calib struct.

4d. Persist

```
p = tfp.io.saveCalibration(calib, 'dmd_affine', config);
tfp.io.updateConfigCalibrationPath('configs/real.yaml', 'session', 'calibration_file', p);
```

- [ ] residual < 1 px; signs verified; config points at the saved file
- [ ] **Record the fitted µm/px scale** — compare against the (regenerated) handoff §5 values. Disagreement > a few % ⇒ something is installed differently (handoff §3 lists the suspects).

---

5. Z ruler bringup (the MP-285 as the common axis)

Everything axial — SLM defocus AND ETL planes — is measured against one objective-z ruler so the composition is a direct substitution. Pick a backend (`zstage.backend` in the config):

- `relay` **(recommended, no recabling):** on the imaging PC (inside the ScanImage MATLAB, where `hSI` lives): `matlab addpath('<repo>/scripts/imaging_pc_setup'); si_motor_helper` ⚠ VERIFY on first use: `hSI.hMotors.motorPosition` is the property this helper reads/writes on SI2019bR0, and assignment blocks until the move completes (see the header of `si_motor_helper.m`).
- `mp285` **(direct serial):** flip the manual switch box toward the DAQ PC and complete `docs/mp285-protocol-audit.md` first — item #1 (µsteps/µm, default 25) scales every z-calibration.
- `manual`**:** zero new hardware; the calibration prompts you to turn the knob.

Smoke test from the DAQ PC:

```
z = tfp.hardware.makeZStage(config);   % + connect for relay backend
z0 = z.getPositionUm(); z.moveRelativeUm(+10);
assert(abs(z.getPositionUm() - z0 - 10) < 0.5); z.moveToUm(z0);
```

- [ ] move/read round-trip good to < 0.5 µm
- [ ] **Sign convention checked**: +z must move focus DEEPER into the sample (the `ZStage` contract). If inverted on this mount, fix it in the backend (one place) — note it in the audit doc.

---

6. Z calibration — indirect method (PRIMARY)

Target: the same thin fluorescent film. **Power:** ~5 mW (single small spot — far under every cap). **Prerequisite:** `config.slm.m_relay` set to your best value from the optics model ⚠ VERIFY — there is no handoff key yet; a wrong value shows up as the fitted slope missing 1.0, which is exactly what this calibration measures.

6a. SLM defocus → physical µm

```
dmd  = tfp.hardware.DLP650LNIR_DMD(config.dmd);
slm  = tfp.hardware.makeModulator(config);      % 'remote' backend; server must be up
cam  = tfp.hardware.BaslerSubstageCamera(); cam.initialize(config.camera);
z    = tfp.hardware.makeZStage(config);

slmCal = tfp.calibration.calibrateSlmDefocus(dmd, slm, cam, z, config);
% defaults: dzCmdUm = -100:25:100, spot r=8 px at chip centre, 5 mW,
% per-dz through-focus sweep ±30 µm in 5 µm steps around the expected focus.
pSlm = tfp.io.saveCalibration(slmCal, 'slm_defocus', config);
tfp.io.updateConfigCalibrationPath('configs/real.yaml', 'slm', 'calibration_file', pSlm);
```

What it does per dz command: advance the SLM, sweep the objective through focus, find the stage z where the spot on the film is sharpest, then fit `zPhys = slope·dzCmd + b`.

Good result: `slmCal.fit.r2 > 0.98` and `|slope| ≈ 1.0` (dz commands are defined in sample-µm). Slope far from 1 ⇒ `m_relay` / objective / immersion mismatch — **the fit is the calibration**; fix the config for

interpretability but the fitted slope is what gets used. Repeat per objective when you swap (`objective.name`: `nikon10x045` | `olympus20x` | `nikon16x` | `avocado10x`).

6b. ETL planes → physical µm

Imaging PC: configure the real experiment's fastZ stack (`hStackManager.numSlices = 3`, your plane spacing), confirm with `probe_scanimage_config('3d')`, and free-run. Then:

```
etlCal = tfp.calibration.calibrateEtlPlanes(z, config, struct( ...
    'planeBrightnessFcn', @myPlaneBrightness));   % see below
pEtl = tfp.io.saveCalibration(etlCal, 'etl_planes', config);
```

The measurement: step the objective (–40…+40 µm); each ETL plane's mean brightness peaks when the film sits at that plane's focal depth. `planeBrightnessFcn` returns the 1×nPlanes brightness after each step — either live (per-plane channel means relayed from the imaging PC) or offline (operator grabs one small stack per step; a wrapper reads the means). The offline route is slower but needs zero new plumbing on day one.

6c. Compose — the ETL↔SLM tag

```
zcal = tfp.calibration.composeZCalibration(slmCal, etlCal);
pZ = tfp.io.saveCalibration(zcal, 'z_composed', config);
tfp.io.updateConfigCalibrationPath('configs/real.yaml', 'etl', 'plane_calibration_file', pZ);
zcal.dzCmdForPlane   % the SLM command that focuses stim at each imaging plane
```

- [ ] slope fit $r^2 > 0.98$, recorded
- [ ] plane depths recorded (spacing should match what you set in ScanImage)
- [ ] `etl.plane_calibration_file` written back

---

7. Direct verify — burn/bleach marks (and the tilt SIGN)

This mirrors the lab's 3D-SHOT two-step protocol and cross-checks §6. It also resolves the one number the indirect method cannot: **the sign of the tilt gradient** (`threeD.depth_gradient_sign`).

Target: thick fluorescent slab.

1. Make the marks — try burn, fall back to bleach if no mark forms (the open question of whether the DMD arm reaches burn threshold): `matlab daq = tfp.hardware.NI6323_DAQ(config.daq); ledger = tfp.calibration.markFluorescentSlab(dmd, slm, daq, config, ... struct('mode', 'burn')); % defaults: dz −40:20:40, 50 mW, 0.5 s % no visible mark? -> struct('mode', 'bleach') (10 mW, 30 s dwell)` Grid position encodes which dz made each mark — no bookkeeping ambiguity.

2. Imaging PC: acquire an ETL z-stack over the marked region; get the pixel data to the DAQ PC as an array (frames plane-interleaved).
3. Localize + verify: `matlab found = tfp.calibration.locateMarksInStack(stack, ledger, calib, ... struct('nPlanes', config.etl.n_planes, 'zcal', zcal)); report = tfp.calibration.verifyZCalibration(zcal, found);` **PASS** = every mark's darkest plane matches the indirect prediction. Disagreement suspects: `m_relay` sign, ETL plane order, stack interleave.
4. **Tilt sign:** additionally burn/bleach a row of spots at `dz = 0` spanning the dispersion diagonal (the chip's (1,1) direction). In the ETL stack the marks march monotonically across planes — the direction of the march IS the sign. Set `threeD.depth_gradient_sign` (±1) in `configs/real.yaml`. (Magnitude always comes from the handoff at runtime; only the sign is yours.)

- [ ] indirect vs direct agree; `depth_gradient_sign` recorded

---

8. First-light 2D photostimulation experiments

Config: `hardwareKind: real`, `slm.enabled: true` with the SLM **blanked or on dz = 0** (2D experiments don't advance it). Fluorescent film or slab first; then ChRmine+GCaMP tissue. Start every session:

```
tfp.util.safetyChecks('arm');
config = tfp.io.loadConfig('configs/real.yaml');
```

Real-target flow for each experiment: draw integration ROIs in ScanImage → run the experiment on the DAQ PC (it listens on 3045) → run `si_send_rois` on the imaging PC.

8a. Power-response curve (key variable #1)

```
result = tfp.experiments.exp_power_curve(config, 'day1_powercurve');
```

Sweeps power at one representative target. Record: threshold power, saturation power, the working point for PPSF runs (typically ~2× threshold).

8b. Lateral PPSF (key variable #2)

```
result = tfp.experiments.exp_ppsf_lateral(config, 'day1_ppsf_lat');
% or exp_ppsf_2d for the full 2D offset-grid heatmap
```

Fraction of responses vs distance of the stim spot from the target cell. Record: lateral FWHM (μm). This is the single-cell-resolution claim — compare against cell spacing (~15–20 μm).

8c. Rapid sequential switching (key variable #3 — the DMD's advantage)

```
result = tfp.experiments.exp_rapid_sequential(config, 'day1_rapidseq');
```

Multiple targets activated in fast succession. Record: shortest ISI with reliable per-target responses. (Pattern-rate headroom was already benchmarked electrically: `scripts/photodiode_timing_tests/runDMDTimingSweep.m`.)

- [ ] power curve, lateral PPSF FWHM, min reliable ISI recorded

---

9. Axial PPSF via SLM remote focus (key variable #4)

The remote-focus replacement for pseudo-axial objective translation:

```
result = tfp.experiments.exp_axial_ppsf(config, 'day2_axial');
% defaults dz = [0 ±5 ±10 ±20 ±30 ±50] µm; override via config.axialPpsf.dzUm
```

With `slm.enabled: true` the experiment auto-uses the sequence path: one prepared mask stack (unique dz values), per-trial group advance + LC settle, no pixel masks over the wire.

Record: **axial PPSF FWHM** — response vs dz around a target cell. Expected order: ~30 µm (handoff rev 4 says 32.6 µm at a 12 µm target — provisional until the f7=300 regen). Also confirm response at dz = ±50 µm is strongly suppressed (off-target protection above/below).

- [ ] axial FWHM recorded; symmetric about the fitted $dz_0 \approx 0$

---

10. 3D ensemble photostimulation (the full system)

Prerequisites: §6c composed z-calibration in the config; `si_motor_helper` free; ETL free-running the 3-plane stack; `probe_scanimage_config('3d')` PASSes.

1. Imaging PC: draw ROIs on cells across all planes; the 3D-mode `si_send_rois` sends Nx4 `[x y planeIdx zUm]` automatically when `numSlices > 1`.
2. DAQ PC: `matlab config.threeD.enabled = true; % or use a 3D session config result = tfp.experiments.exp_3d_ensemble(config, 'day3_3d');` What happens: targets are binned by required SLM defocus (dz = z_target − sign·gradient·x_disp — cells across one DMD FOV span ~3 imaging planes because of the tilted excitation plane); the SLM sequence is prepared once in group order; per depth group the Sequencer advances the SLM (TTL or network + 3.4 ms settle), displays that group's multi-spot pattern, stimulates; ScanImage free-runs volumes throughout; frames are plane-tagged post-hoc from the frame clock (gap-aware — a dropped frame cannot silently shift the plane assignment).
3. Read `result.summary` (per depth group) and check `result.perFrame` plane-confidence: all `"exact"` is the goal; `"corrected"` is usable; `"quarantined"` trials need scrutiny.

Validation experiment (do this before biology): fake-cell-style film test — targets in 3 planes, confirm each group's response appears in its own plane's frames only. Then in tissue: record cross-plane response

contrast (the 3D off-target measure) and rapid group-switching timing (group advance + settle should add ≤ ~5 ms per depth change).

- [ ] per-plane responses land in the right planes
- [ ] 3D session with ≥ 2 depth groups completed end-to-end

---

11. The record sheet — key variables to log

#	Variable	Measured by	Expected (provisional, rev-4 handoff)
1	Lateral µm/px (both axes)	§4 affine fit	handoff §5 (rescales at f7=300 regen)
2	Volts → mW curve	§3 power sweep	—
3	SLM defocus slope (µm/µm)	§6a fit	≈ 1.0, $r^2 > 0.98$
4	ETL plane depths (µm)	§6b	your configured spacing
5	Indirect vs direct z agreement	§7 verify	all marks agree
6	Tilt gradient **sign**	§7.4 mark row	±1 → config
7	Power threshold / saturation	§8a	—
8	**Lateral PPSF FWHM**	§8b	— (grant figure)
9	Min reliable ISI (switching)	§8c	— (grant figure)
10	**Axial PPSF FWHM**	§9	~32.6 µm (rev 4)
11	Cross-plane response contrast	§10	strong own-plane preference
12	SLM group-advance overhead	§10 logs	~settle (3.4 ms) + link latency

---

Appendix A — troubleshooting quick hits

- **`tfp:hardware:MeadowlarkSLM:replyTimeout`** — `slm_server` not running / wrong `slm.host` / firewall. Server prints every command it receives.
- **`tfp:hardware:BlinkSLM:sdkNotVendored`** — the Blink audit (§2) isn't done; server runs dry until then.
- **`tfp:util:assertSlmPowerSafe:slmAlignmentCapExceeded`** — you asked for a near-uniform pattern above 44 mW with the SLM in the path. Lower power for alignment patterns; sparse targets are exempt by design.
- **`tfp:hardware:DMD:onFractionExceeded`** — pattern > 50% ON. Not overridable.
- **Through-focus sweep finds no spot** (`tooFewPoints`) — film out of range, exposure too low, or search window missed focus: widen `zSearchHalfUm`, check `sweepOptions.threshFrac` (default 0.3), add a `backgroundFrame` (no-stim snap) for dim defocused spots.
- **Fitted defocus slope far from 1** — `slm.m_relay` wrong (quadratic effect), wrong `objective.name`, or immersion mismatch. The fit is still valid.

- **Plane confidence "quarantined"** — non-integer gap in the frame clock: check the DI wiring/rate, re-run; don't trust plane tags past the gap.
- **ROIs arrive Nx2 in 3D mode** — imaging PC wasn't in the multi-plane stack when `si_send_rois` ran (`numSlices` was 1).
- **`probe_scanimage_config` FAILs on numSlices** — run it as `probe_scanimage_config('3d')` for 3D sessions.

Appendix B — file index (what implements each step)

Step	Code
SLM masks (shared engine)	`src/+tfp/+slm/` (`computeDefocusMask`, `computeDefocusStack`, `buildMaskSpec`)
SLM server / proxy / real driver	`scripts/slm_pc_setup/slm_server.m`, `tfp.hardware.MeadowlarkSLM`, `tfp.hardware.BlinkSLM`
Objectives / optics	`tfp.optics.objectives`, `buildDefocusSys`, `dmdToDispersionUm`
Z ruler	`tfp.hardware.ZStage` + `MP285ZStage` / `RelayZStage` / `ManualZStage`, `scripts/imaging_pc_setup/si_motor_helper.m`
Lateral calibration	`tfp.calibration.alignDMDtoCamera`, `crossRegisterScanImage`, `composeCalibration`, `verifyScanFieldComposition`
Z calibration (indirect)	`tfp.calibration.calibrateSlmDefocus`, `calibrateEtlPlanes`, `composeZCalibration`, `throughFocusSweep`
Z calibration (direct)	`tfp.calibration.markFluorescentSlab`, `locateMarksInStack`, `verifyZCalibration`
Calibration IO	`tfp.io.saveCalibration`, `loadCalibration`, `updateConfigCalibrationPath`, `loadCalibrationOrIdentity`
3D experiment	`tfp.experiments.exp_3d_ensemble`, `TrialSequence.generate3DEnsemble`, `Sequencer` (slm/threeD), `tfp.io.assignFramePlanes`, `alignTrialsFreeRun`
2D experiments	`exp_power_curve`, `exp_ppsf_lateral`, `exp_ppsf_2d`, `exp_rapid_sequential`, `exp_axial_ppsf`
Safety	`tfp.hardware.DMD.assertPatternsSafe` (50% ON), `tfp.util.assertSlmPowerSafe` (44 mW LC cap), `tfp.util.safetyChecks`
Audits / gates	`docs/blink-api-audit.md`, `docs/mp285-protocol-audit.md`, `docs/PORTS.md`